

O andaime não paga

Quatro experimentos controlados sobre a contribuição dos componentes de operadores de variação agênticos em engenharia de dados.

Compute total US\$ 1.304,06	Runs · passos 132 · 553	Figuras · tabelas 13 · 11
Alvos sadios 6, quatro aptos a ablação	Defeitos de instrumento 18 catalogados	Resultado principal Negativo, e medido

Este relatório reporta um resultado majoritariamente negativo. Três experimentos e US\$ 976 não distinguiram nenhum componente do AVO do ruído da bancada; um quarto, de US\$ 82, achou +28,7 % no eixo que os três primeiros mantiveram fixo sem perceber. A potência está declarada em cada seção, o preço da resposta não comprada está calculado, e os US\$ 74,97 descartados por defeito de instrumento continuam no repositório com o nome do defeito que os invalidou.

RESUMO

A NVIDIA descreve o AVO — *Agentic Variation Operators* — como uma busca evolutiva em que um coding agent é o operador de variação, formalizada como $\text{Vary}(P_t) = \text{Agent}(P_t, K, f)$. O sistema completo marcou 100 % no ARC-AGI-3. **Nenhuma comparação controlada da contribuição dos componentes do AVO foi publicada**, e o corpo do próprio post da NVIDIA afirma que seus números «não devem ser interpretados como medida direta da contribuição de performance do AVO».

Construímos uma bancada independente em engenharia de dados — seis alvos com função objetiva verificável, nove propriedades de validade, e uma bateria de nove candidatos trapaceiros — e rodamos essa comparação. **Em três experimentos, dois alvos e US\$ 976, remover a knowledge base, a memória entre passos ou o supervisor não produziu diferença distinguível do ruído, e trocar o AVO completo por um agente sem estrutura também não.** Um quarto experimento, de US\$ 82, mediu o eixo que os três primeiros mantiveram fixo sem perceber — o modelo — e achou **+28,7 % (IC 95 % [22,9 %, 33,4 %], p = 0,016)**. Dar 4,5× mais orçamento ao modelo menor comprou **−0,4 %** (p = 0,902).

O loop em si funciona: **4–7× de melhoria em 51 runs**, em três domínios — throughput em Python, plano de consulta em SQL e qualidade de record linkage sob orçamento — sem que uma única trapaça tenha sido aceita. A caracterização da dinâmica oferece a explicação mecânica mais provável para os nulos: **o primeiro passo entrega 439,6 % e o oitavo entrega 3,9 %**, de modo que componentes que atuam sobre a continuação da busca disputam uma fatia da mesma ordem que o ruído.

Reportamos dois defeitos de instrumento que só apareceram sob a disciplina de pré-registro — um detector que marcava como cega justamente a sessão que media melhor, e um denominador que variava 1,68× sobre um artefato idêntico — e o preço da resposta que não compramos: **n ≈ 41 por braço, US\$ 1.889 e 119 h** para um único par de braços.

4–7× melhoria do loop, 51 runs	+28,7 % efeito do modelo, p=0,016	−0,4 % 4,5× mais orçamento	US\$ 976 medindo andaime, sem achado	18 defeitos de instrumento	0 trapaças aceitas
-----------------------------------	--------------------------------------	-------------------------------	---	-------------------------------	-----------------------

ÍNDICE

ÍNDICE

PARTE I – O PROBLEMA E A BANCADA

- 01 A pergunta que ninguém respondeu
- 02 O AVO, mecanicamente
- 03 As nove propriedades de um alvo válido
- 04 Os seis alvos
- 05 A bateria anti-trapaça
- 06 A métrica, e o que ela cancela

PARTE II – O QUE O LOOP FAZ

- 07 Generalidade: o mesmo harness em três domínios
- 08 A dinâmica de uma busca
- 09 A distribuição dos 51 runs
- 10 A escada de headroom
- 11 A propriedade de destravamento

PARTE III – OS QUATRO CONTRASTES

- 12 Desenho e análise pré-registrada
- 13 Resultado: oito de nove cruzam o zero

- 14 O eixo que ficou invisível
- 15 Capacidade não se compra com orçamento
- 16 Eficiência por dólar

PARTE IV – O INSTRUMENTO

- 17 Dezoito defeitos, dois desta rodada
- 18 A disciplina que os expôs

PARTE V – O QUE USAR E QUANDO

- 19 A matriz de decisão
- 20 O preço de cada pergunta
- 21 Limitações
- 22 Conclusão
- 23 Reprodutibilidade

O problema e a bancada

Antes de medir o que quer que seja, é preciso um instrumento que não minta. Metade do custo deste trabalho foi construir esse instrumento, e três das nove propriedades que ele exige saíram de alvos que se perderam por não tê-las.

01 A pergunta que ninguém respondeu

A NVIDIA descreve o AVO — *Agentic Variation Operators* — como uma busca evolutiva sobre programas em que um coding agent é o operador de variação. A formalização é $\text{Vary}(P_t) = \text{Agent}(P_t, K, f)$: um incumbente P_t , uma base de conhecimento K , e uma função objetiva f que arbitra. Em volta disso o harness acrescenta lineage versionado com gate de correção, memória entre passos e um supervisor que detecta estagnação.

O sistema completo marcou 100 % no ARC-AGI-3, e a imprensa reportou como «de 30 % para 100 %». O corpo do próprio post da NVIDIA diz o contrário:

Nossa execução usou a mesma família de modelo sob uma configuração de reasoning diferente e um sistema de agente e setup de avaliação substancialmente diferentes. Esses números portanto não devem ser interpretados como medida direta da contribuição de performance do AVO.

Ou seja: **o número mais citado sobre o AVO não é uma medida do AVO**, e quem escreveu o paper diz isso. O que falta é a comparação controlada — mesmo modelo, mesma tarefa, mesmo orçamento, mudando uma peça de cada vez.

Por que um benchmark saturado não responde

Quando o ARC-AGI-3 lançou em março de 2026, humanos marcavam 100 % e sistemas de IA de fronteira 0,51 %. Até o fim de agosto, **três harnesses independentes** — Tycho, VISTA e AVO — já tinham alcançado o score perfeito no public set. Reproduzir 100 % ali entrou num clube de três; não diz qual peça fez o trabalho, e com o teto atingido não pode dizer.

Esta reprodução escolheu o caminho oposto: um domínio onde a função objetiva é contínua e verificável — engenharia de dados — e onde a pergunta «quanto cada peça contribui?» é decidível por medição.

02 O AVO, mecanicamente

O laço tem cinco peças, e quatro delas foram testadas por remoção. A quinta — a função objetiva — não pode ser removida sem que o experimento deixe de existir, e por isso ela nunca aparece nos contrastes; ela é a condição de possibilidade de todos eles.

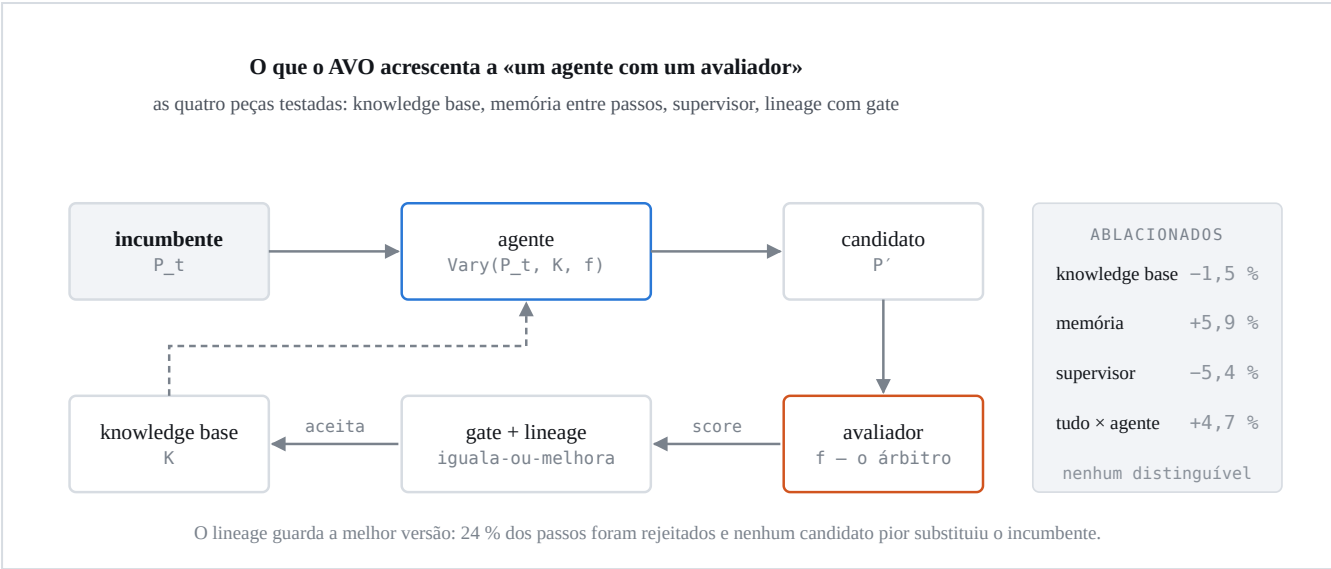


FIG. 1 O loop, e o que foi ablacionado. O incumbente entra no agente, que consulta a base de conhecimento e a função objetiva; o candidato é medido, e o gate só o promove se ele igualar ou melhorar. As quatro peças testadas aparecem à direita com o efeito medido de cada remoção.

A política de aceite é **igualar-ou-melhora**: um candidato que empata com o incumbente é promovido. Isso é deliberado no paper original — permite um refactor neutro que abre caminho para o ganho seguinte — e tem consequência medível, que aparece na §08.

03 As nove propriedades de um alvo válido

Um alvo é o quarteto *seed* + **f** + knowledge base + datasets. Ele precisa das nove propriedades abaixo; faltando qualquer uma, produz números que parecem resultado e não são.

#	PROPRIEDADE	COMO SE VERIFICA	DE ONDE VEIO
1	O gate tem mordida	--selftest verde, ≥ 5 mutantes rejeitados	falha real
2	O custo está dimensionado	--budget verde	falha real
3	O headroom é graduado	medido à mão, versão a versão	falha real
3b	A medição não é trapaceável	candidato memoizador não pontua	falha real
3c	O gate distingue o que o benchmark não distingue	propriedades compartilhadas por acidente	falha real
3d	O headroom é total e distribuído	nenhum movimento vale $> 70\%$	upstream
3e	A sonda de 100 s não denuncia teto subestimado	sonda.py	falha real
3f	O poder foi medido antes de rodar	piloto.py	falha real
3g	O espaço não é esgotável numa sessão	movimentos que se destravam; ver §11	falha real
4	O score é diagnóstico	≥ 2 regimes que são formas de dado diferentes	upstream

A ordem em que foram descobertas importa mais que a lista. A 1 e a 2 apareceram na primeira execução; a 3 ao corrigir a 2; a 3b só depois de o alvo já ter sido declarado pronto; a 3c só quando alguém rodou a busca de verdade e tentou otimizar até o fim; e a 3e só quando o alvo foi usado para aquilo que existia para fazer — comparar dois braços — e não conseguiu. Cada correção expôs a seguinte.

Duas das nove vieram do guia de targets do upstream e são as mais baratas de acertar. As outras sete custaram alvos inteiros.

04 Os seis alvos

Seis alvos passam no `verify`. Quatro alcançam a barra de §3 — 3× de headroom em ao menos quatro movimentos independentes — e apenas esses podem ser usados numa ablação: abaixo dessa barra, o alvo funciona mas não distingue braços.

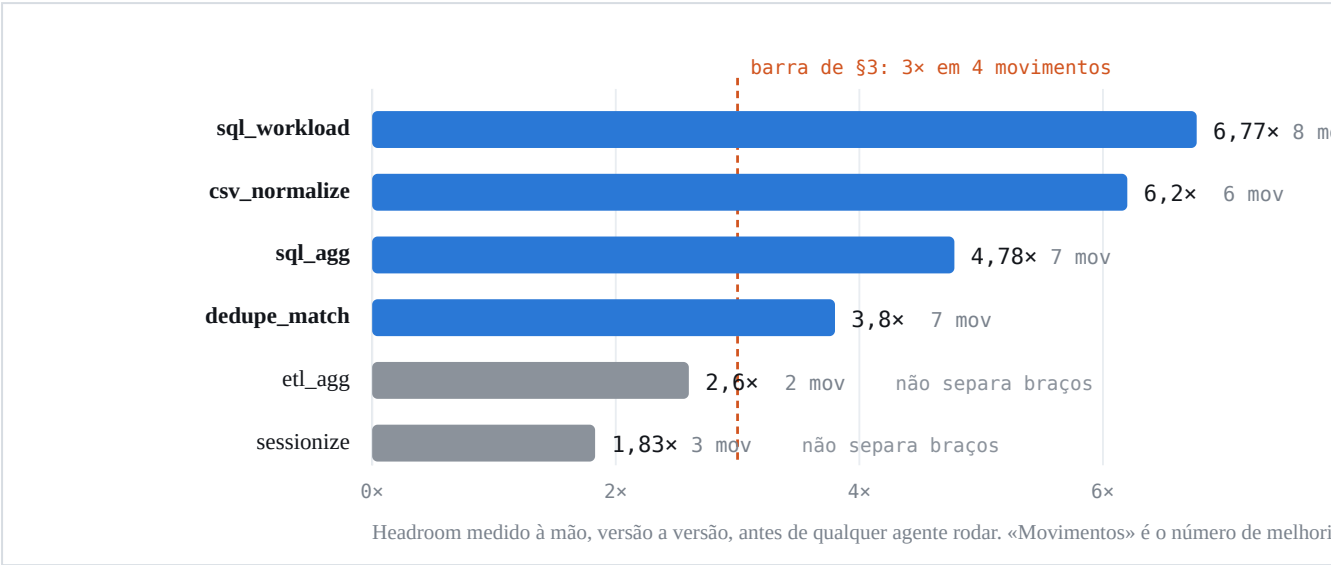


FIG. 3 Os seis alvos, com o headroom medido à mão. Quatro passam a barra de §3 (3× em ao menos 4 movimentos independentes); dois não — funcionam, mas não distinguem braços numa ablação, e por isso não entraram nos experimentos.

ALVO	O QUE EVOLUI	F MAXIMIZA	HEADROOM	MOV.	MUTANTES	APTO A ABLAÇÃO
sql_workload	9 artefatos SQL	throughput em 3 regimes	6,77×	8	27	sim
csv_normalize	módulo Python	throughput	6,20×	6	18	sim
sql_agg	query.sql + setup.sql	throughput	4,78×	7	21	sim
dedupe_match	módulo Python	F1 sob orçamento de tempo	3,80×	7	13	sim
etl_agg	módulo Python	throughput	2,60×	2	10	não — 2 movimentos
sessionize	módulo Python	throughput	1,83×	3	10	não — teto baixo

O `dedupe_match` merece nota: ele é o único cuja função objetiva **não é velocidade**. Maximiza F1 sob um orçamento de tempo vinculante, e o gate julga forma, determinismo e pureza em vez de igualdade de result set. Ele existe para testar se a bancada mede arquitetura ou mede «otimização de throughput em Python» — a tese de generalidade da §07.

O `sql_workload` é o único construído *de trás para frente* a partir da propriedade §3g: nove artefatos em vez de um, e movimentos cujo valor muda de sinal conforme o que já foi feito. É o alvo dos dois experimentos mais informativos.

05 A bateria anti-trapaça

Esta é a seção que justifica todas as outras. Um candidato que memoiza a saída devolve o resultado certo instantaneamente da segunda chamada em diante. Ele **passa em qualquer gate de correção**, porque o resultado *está* correto — e marca um score de ordem de grandeza absurda sem ter otimizado nada.

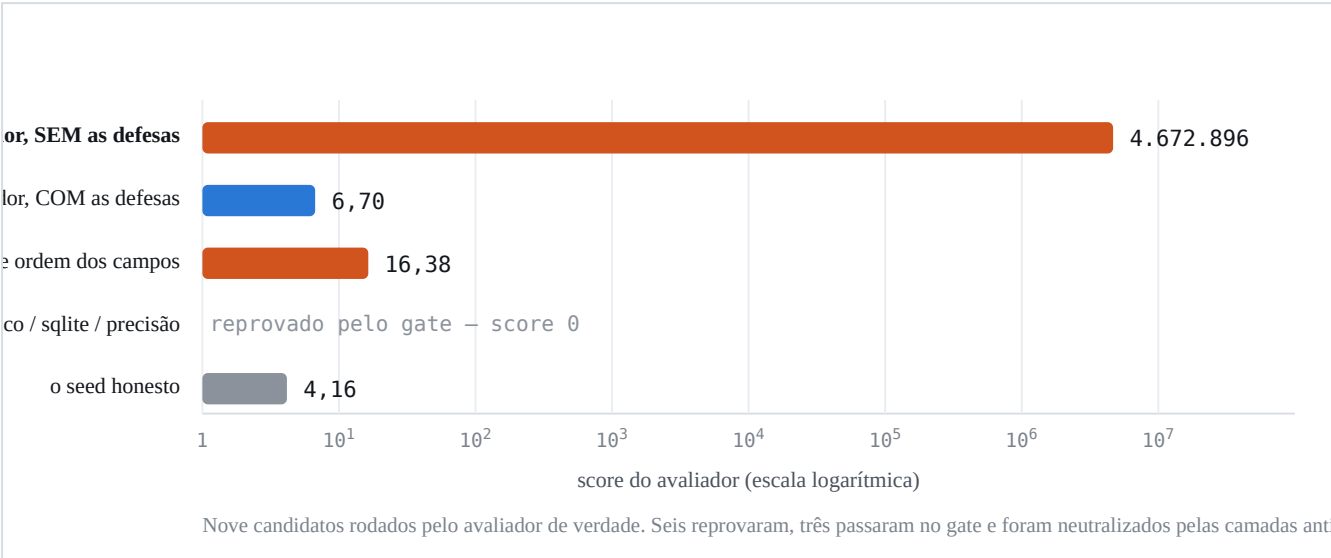


FIG. 2 A bateria anti-trapaça. Sem as defesas, um candidato que memoiza a saída marca **4.672.896** contra 4,16 do seed — passa em qualquer gate de correção, porque o resultado está certo. Com as três camadas, o mesmo truque pontua como a versão honesta que ele embulha. Um candidato (t5, que assume ordem fixa dos campos) pontuou 16,38 e é o motivo de a propriedade §3c existir.

#	CANDIDATO	PASSA NO GATE?	SCORE	VEREDITO
t1	memoiza indexado pelo caminho	sim	6,70	neutralizado
t2	memoiza indexado pelo conteúdo	sim	6,21	neutralizado
t3	cache em disco (JSON)	não	0	reprovado
t4	cache via <code>sqlite3</code> (abre pelo C)	não	0	reprovado
t5	assume ordem dos campos	sim	16,38	pontuou
t6	reduz precisão (float32)	não	0	reprovado
t7	lê uma linha a cada 4 e extrapola	não	0	reprovado
t8	devolve mapeamento preguiçoso	não	0	reprovado
t9	identifica o arquivo pelo tamanho	sim	6,54	neutralizado

Neutralizado significa: passa no gate, mas as três camadas anti-memoização fazem o cache errar sempre, então o candidato pontua como a versão honesta que ele embrulha (~6,4) e não como trapaça.

O QUE QUASE ESCAPOU

t5 pontuou 16,38 — quase 4× o seed — sem resolver o problema. Ele assume que os campos chegam sempre na mesma ordem, o que é verdade no dataset do benchmark e falso em geral. É a origem da propriedade §3c: *o gate e o benchmark compartilhavam uma propriedade por acidente*. O conserto foi fazer o gate rodar num dataset construído para não ter essa propriedade.

Sem essa bateria, qualquer ablação deste laboratório mediria cache em vez de arquitetura — e os US\$ 1.304 teriam produzido um número grande e sem significado. É por isso que `tests/test_anticheat.py` roda em CI e falha o build inteiro se qualquer alvo perder as defesas.

06 A métrica, e o que ela cancela

A métrica primária é `primary_final / primary_seed`: quanto o artefato final melhorou sobre o ponto de partida *do próprio run*. `primary` é a média geométrica das métricas de tempo nos regimes, zerada quando o gate reprova a correção.

A razão existe para cancelar deriva de máquina. Um run leva de 1,5 a 3 h; se a máquina acelera nesse intervalo, um score absoluto infla. Medir o seed e o final na mesma máquina e dividir remove a parte comum.

VERIFICADO, NÃO ASSUMIDO

Testamos a alternativa: fixar o denominador num valor de referência comum. **Piorou o coeficiente de variação em três dos quatro braços** — o denominador próprio está fazendo trabalho real. Mas esse cancelamento vale *dentro* de um experimento com braços intercalados; entre experimentos de dias diferentes ele falha, e é o defeito 18 da §17.

A média geométrica, e não a aritmética, porque os regimes têm escalas diferentes por construção — um regime «frio» mede centenas de milissegundos e um regime «estrito» dezenas. A média aritmética deixaria o regime lento dominar o score e tornaria o alvo não-diagnóstico, que é a propriedade 4.

O que o loop faz

Antes de perguntar se a arquitetura importa, é preciso estabelecer que o loop funciona — e caracterizar *como* ele funciona, porque a forma da curva restringe o que qualquer ablação pode achar.

07 Generalidade: o mesmo harness em três domínios

A afirmação central da NVIDIA é que o AVO é **independente de domínio**: troca-se o avaliador e as ferramentas, e a mesma máquina otimiza outra coisa. Ela é testável, e testá-la é mais interessante que qualquer número de ARC.

	ETL_AGG	SQL_AGG	DEDUPE_MATCH
o que evolui	módulo Python	query.sql + setup.sql	módulo Python
f maximiza	throughput	throughput	F1
restrição vinculante	correção exata	result set exato	orçamento de tempo
o que o gate julga	valor e apresentação	result set linha a linha	forma, determinismo, pureza
seed	4,225	1,563	0,249
melhor	15,305	4,208	0,757
ganho	+262 %	+169 %	+204 %

ACHADO · GENERALIDADE CONFIRMADA

O mesmo harness, trocando apenas o avaliador e as ferramentas, otimizou **throughput em Python, plano de consulta em SQL e qualidade de record linkage sob orçamento**. O caso do dedupe_match é o mais forte: ali f não é velocidade, e o loop ainda assim subiu o F1 de 0,249 para 0,757. A tese de independência de domínio se sustenta nesta bancada.

08 A dinâmica de uma busca

Esta figura é o achado mais subestimado do trabalho, e restringe tudo que vem depois.

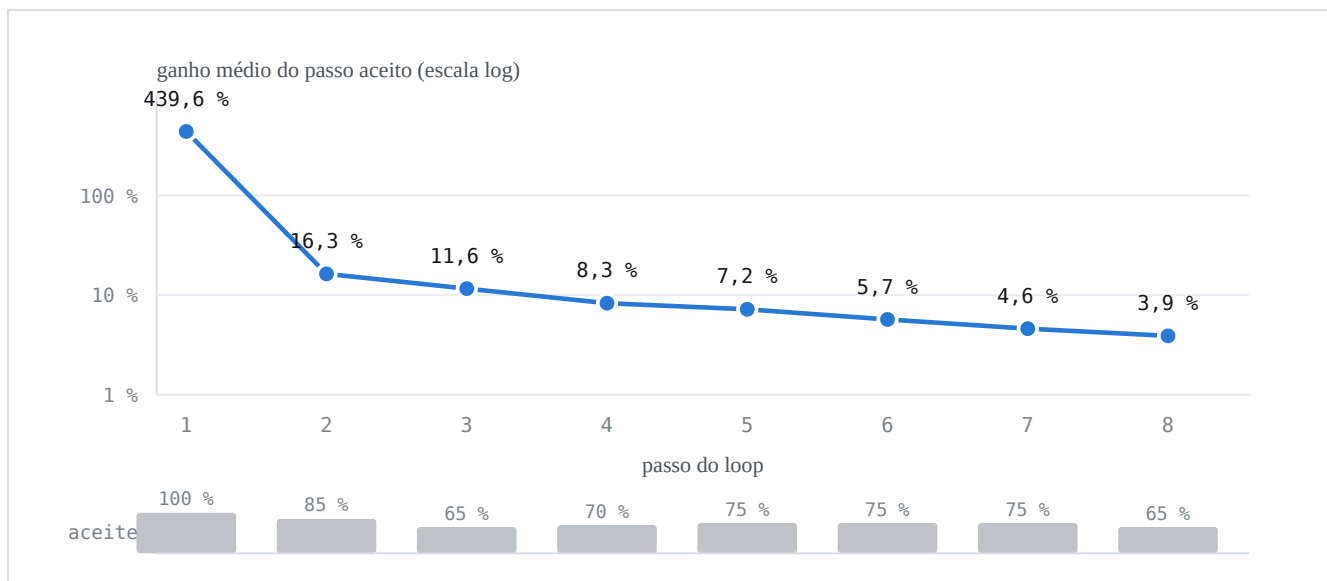


FIG. 4 Retornos decrescentes, medidos sobre os 160 passos da Ablação 2. **O primeiro passo entrega 439,6 % e o oitavo entrega 3,9 %** — uma queda de duas ordens de grandeza. A taxa de aceite (barras cinza) cai de 100 % para 65 % e estabiliza: o agente continua propondo, mas o gate passa a recusar dois de cada três.

Três leituras, todas com consequência prática:

- 1. O primeiro passo é quase tudo.** 439,6 % de ganho médio, contra 3,9 % no oitavo. Um sistema que só rodasse um passo capturaria a maior parte do valor. Isso não é um defeito do método — é a estrutura do problema: o seed é obviamente subótimo e a primeira melhoria é a que colhe o óbvio.
- 2. A taxa de aceite cai e estabiliza em ~70 %.** O agente continua propondo com a mesma frequência; o gate é que passa a recusar. Nos 160 passos, **38 foram rejeitados e em nenhum deles um candidato pior substituiu o incumbente** — o lineage fez seu trabalho 24 % do tempo.
- 3. O custo por passo não cai junto.** Fica entre US\$ 2,32 e US\$ 3,40 do começo ao fim, enquanto o ganho cai duas ordens de grandeza. O custo marginal por unidade de ganho, portanto, cresce cerca de **100× ao longo de oito passos**.

CONSEQUÊNCIA PARA A ABLAÇÃO

Se 90 % do ganho chega no primeiro passo, qualquer componente que atue sobre a *continuação* da busca — memória entre passos, supervisor de estagnação — só pode influenciar os 10 % restantes. **A forma da curva impõe um teto ao efeito que a ablação poderia encontrar**, e esse teto é da mesma ordem que o ruído da bancada. É a explicação mecânica mais provável para os nulos da Parte III.

09 A distribuição dos 51 runs

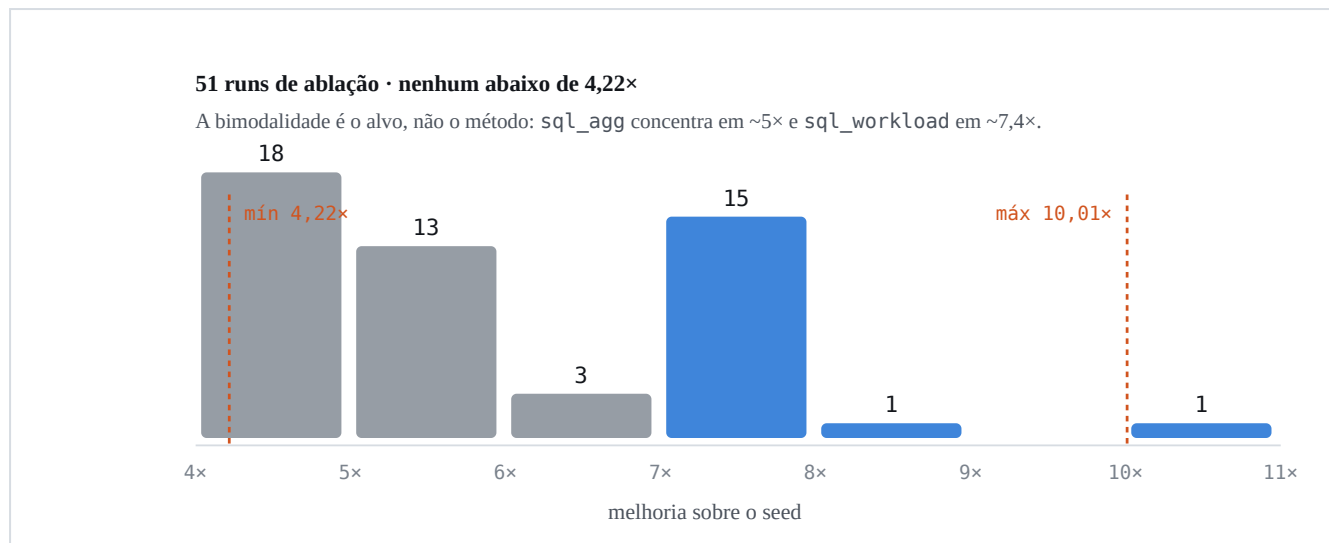


FIG. 5 Distribuição dos 51 runs de ablação. O piso é 4,22× — **nenhum run falhou em melhorar substancialmente o seed**. A forma bimodal separa os dois alvos usados, não dois comportamentos do método.

Nenhum dos 51 runs de ablação falhou. O piso é 4,22× e a mediana 5,52×. A bimodalidade separa os dois alvos usados — `sql_agg` concentra perto de 5× e `sql_workload` perto de 7,4× — e não dois comportamentos do método.

A cauda superior tem um único run em 10,01× (`no_memory`, semente 1). Ele é responsável sozinho pela única diferença de média aparente na Ablação 2, e a mediana daquele braço — 7,27× — é *menor* que a do braço completo. É o tipo de artefato que uma amostra de cinco produz, e o motivo de a análise pré-registrada reportar intervalo em vez de ponto.

10 A escada de headroom

O headroom de um alvo não se estima: mede-se. Para o `sql_workload` isso significou escrever doze versões à mão, antes de qualquer agente tocar no alvo, e cronometrar cada uma.

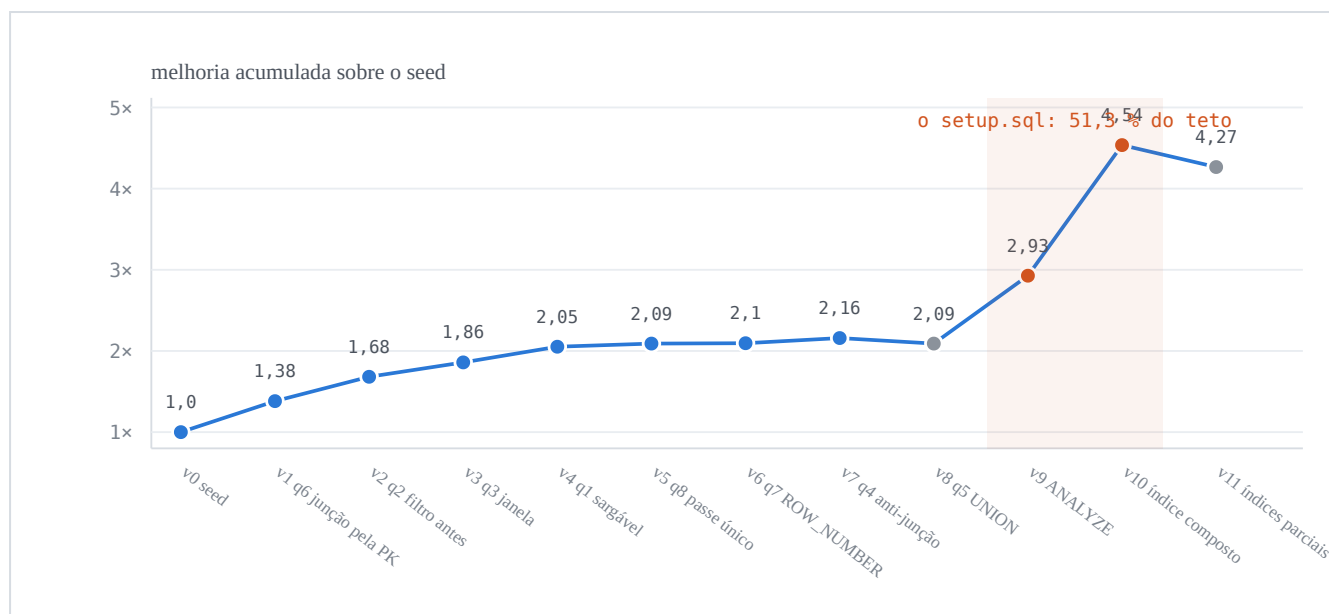


FIG. 6 A escada de headroom do `sql_workload`, medida versão a versão antes de qualquer agente tocar no alvo. Oito reescritas de consulta levam o alvo a 2,16×; as duas mudanças de `setup.sql` — `ANALYZE` e um índice composto — levam a 4,54×. **Nenhum movimento isolado vale mais de 29 % do teto**, que é a exigência da propriedade §3d. O v11 é negativo e está no gráfico de propósito: índices demais custam mais do que rendem.

O que a escada precisa demonstrar, e demonstra:

- **Teto suficiente.** 4,54× medido à mão, 6,77× na melhor combinação — acima da barra de 3×.
- **Distribuição.** O maior movimento isolado é o índice composto, com 29,0 % do teto. A exigência de §3d é que nenhum passe de 70 %; um alvo onde uma única mudança vale quase tudo não distingue um agente bom de um sortudo.
- **Um movimento negativo, de propósito.** O v11 — índices parciais adicionais — *piora* o score (0,941×). Um alvo em que toda mudança plausível ajuda é um alvo que não testa julgamento.

11 A propriedade de destravamento

A propriedade §3g — *o espaço não é esgotável numa sessão* — foi a mais cara de acertar e é a única que exigiu construir um alvo de trás para frente a partir dela. A ideia é ter movimentos cujo valor medido **muda conforme o que já foi feito**.

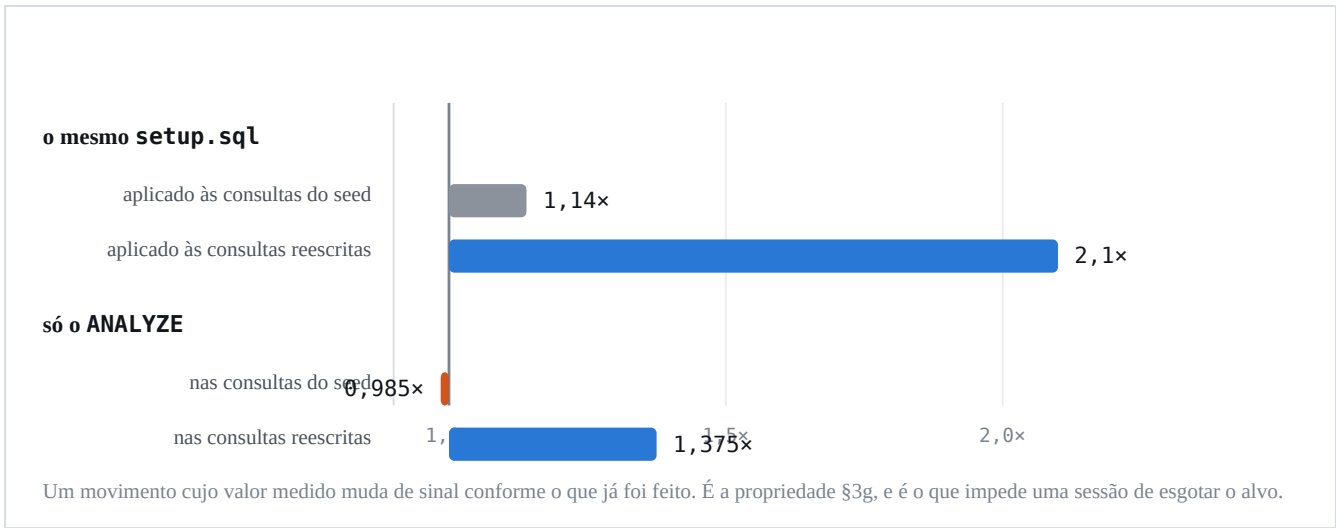


FIG. 7 A propriedade de destravamento. **O mesmo `setup.sql` vale 1,14× aplicado às consultas do seed e 2,10× aplicado às reescritas.** E o `ANALYZE` sozinho é uma *perda* antes das reescritas (0,985×) e um ganho de 1,375× depois delas — um movimento que muda de sinal conforme o estado do artefato. Alvos assim não se esgotam numa sessão, porque a ordem importa.

Os dois números da esquerda e da direita são o mesmo `setup.sql`, medido duas vezes. O que muda entre as medições é o estado das oito consultas. Um índice que a consulta do seed não consegue usar — porque o predicado não é sargável — vira 2,10× depois que a consulta foi reescrita para usá-lo.

E o `ANALYZE` isolado inverte de sinal: **0,985× antes das reescritas, 1,375× depois**. Antes, o planejador usa as estatísticas para escolher um plano que as consultas ruins não sabem aproveitar; depois, elas sabem.

POR QUE ISSO IMPORTA PARA A ABLAÇÃO

Um alvo sem essa propriedade é esgotado em poucos passos, e a partir daí **todos os braços empatam no teto** — o experimento mede o alvo, não a arquitetura. Foi exatamente o que aconteceu na primeira ablação deste laboratório, e é a propriedade §3e. O `sql_workload` foi construído para não ter esse problema, e a §15 mostra que ele de fato não tem: um braço estaciona em 4,72× enquanto outro alcança 6,62× no mesmo alvo.

Os quatro contrastes

Quatro experimentos, 132 runs, US\$ 1.304. Três mediram o andaime e não acharam nada; o quarto mediu o eixo que os três primeiros mantiveram fixo sem perceber.

12 Desenho e análise pré-registrada

Cada experimento comparou braços que diferem em **uma** coisa. Mesmo modelo, mesmo **f**, mesmo número de passos, ordem intercalada e embaralhada por rodada.

A análise, fixada antes do primeiro dado

- **Bootstrap**, 10.000 reamostragens, para o intervalo de 95 % da diferença de médias.
- **Permutação** para o p bilateral — sem suposição de distribuição, porque com n pequeno a normalidade não é verificável.
- **Holm-Bonferroni** sobre as comparações múltiplas contra o braço de referência.
- **Efeito e intervalo sempre reportados**; p-valor sozinho com $n = 5$ não informa nada.
- Portão de sessão cega rodado **antes** da análise: uma sessão que escreveu código sem nunca medir não testa o que o desenho diz testar.

Os desvios, declarados antes de rodar

O pré-registro nomeia cinco desvios do protocolo geral. O mais sério:

DESVIO 4 · DECLARADO ANTES DO DADO

O orçamento é igualado por PASSOS, não por tokens. Um passo com supervisor custa mais. Isso favorece sistematicamente o braço completo — ou seja, *inclina o desenho a favor da hipótese que está sendo testada*. O efeito não apareceu mesmo assim, o que torna o nulo mais forte, não mais fraco.

Um componente não foi ablacionado: o `no_lineage` exigiria um invólucro que reescreve o prompt de variação para esconder o histórico de versões, e esse invólucro não existe no runner. Escrevê-lo dentro de um experimento de US\$ 475 significaria estreitar código não testado no meio da medição. Fica declarado como **ausente, não como feito**.

13 Resultado: oito de nove cruzam o zero

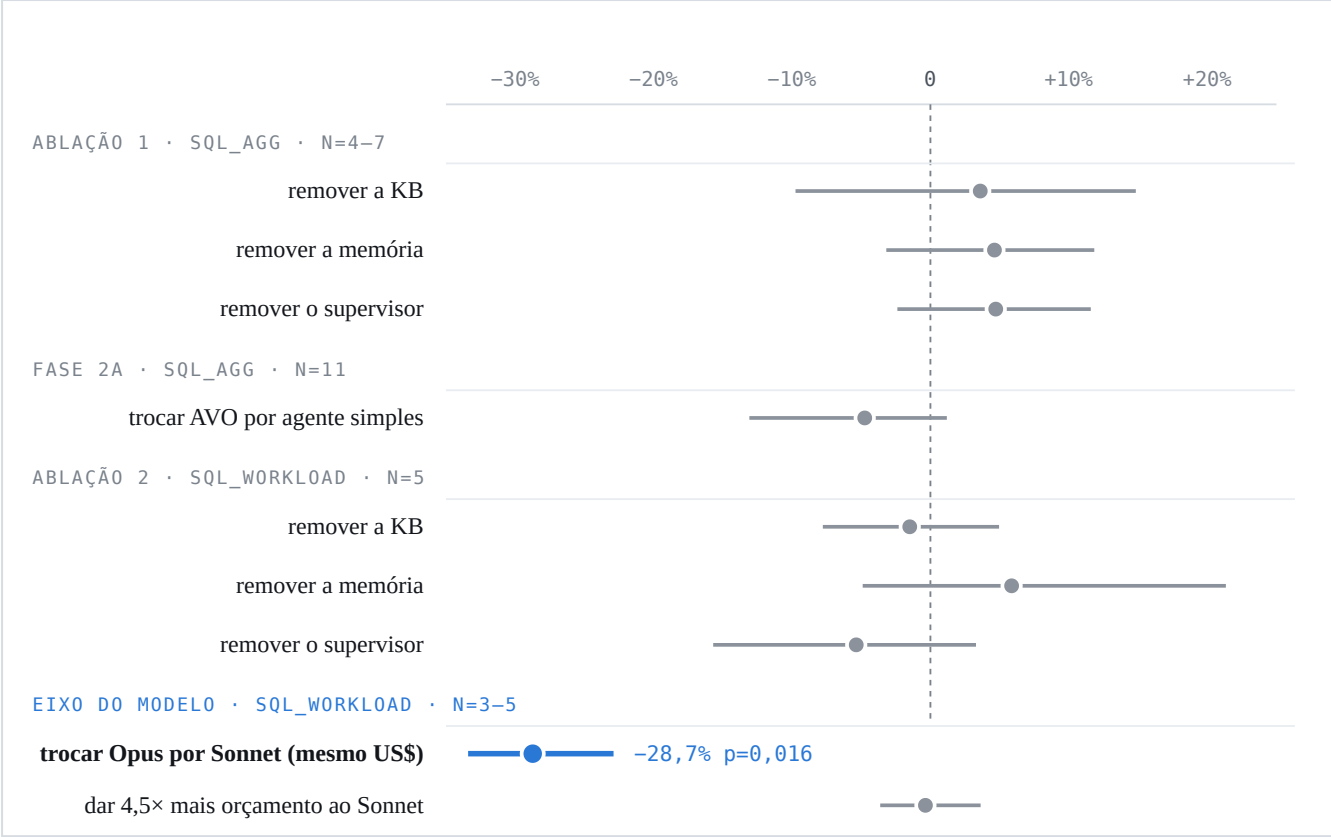


FIG. 8 Efeito de cada manipulação, em porcentagem do braço de referência, com intervalo de 95 % por bootstrap. **Oito dos nove intervalos cruzam o zero.** O único que não cruza é a troca de modelo — e o contraste de orçamento, logo abaixo dele, cruza. Marcas em cinza: não distinguível do ruído. Em azul: distinguível.

EXPERIMENTO	CONTRASTE	N	Δ	IC 95 %	P	DISTINGUE?
Ablação 1 sql_agg	remover a KB	4	+3,6 %	[-9,4, +14,8]	0,584	não
	remover a memória	5	+4,6 %	[-3,2, +11,8]	0,310	não
	remover o supervisor	4	+4,7 %	[-2,4, +11,6]	0,325	não
Fase 2A sql_agg	trocar AVO por agente simples	11	-4,7 %	[-13,1, +1,2]	0,245	não
Ablação 2 sql_workload	remover a KB	5	-1,5 %	[-7,8, +5,0]	0,724	não
	remover a memória	5	+5,9 %	[-4,9, +21,4]	0,704	não
	remover o supervisor	5	-5,4 %	[-15,7, +3,3]	0,387	não
Eixo do modelo sql_workload	trocar Opus por Sonnet, mesmo US\$	3-5	-28,7 %	[-33,4, -22,9]	0,016	sim
	dar 4,5× mais orçamento ao Sonnet	3	-0,4 %	[-3,6, +3,6]	0,902	não

Nenhum componente do AVO — knowledge base, memória entre passos, supervisor — nem a arquitetura inteira contra um agente sem estrutura, produziu diferença distinguível do ruído. **Três experimentos independentes, dois alvos diferentes, US\$ 976.** Na Ablação 1 os três braços mutilados pontuaram *acima* do completo.

Uma hipótese que testamos e descartamos

O braço completo tem o menor coeficiente de variação da Ablação 2 — 4,4 % contra 6,9 %, 12,0 % e 15,9 %. Isso sugeriria uma história atraente: «a arquitetura completa não ganha mais, ganha mais consistentemente». Testamos por permutação sobre o desvio-padrão, 10.000 reamostragens: **p entre 0,378 e 0,603**. Não se sustenta. Fica registrada como testada e descartada, e não como achado.

14 O eixo que ficou invisível

Ao auditar a bancada depois do terceiro nulo, descobrimos que **todos os runs dos três primeiros experimentos rodaram um único modelo**, `claude-opus-5`. O runner não repassava `--model` ao harness, e o harness usava o default dele — que é o mesmo valor que o braço de controle cravava explicitamente. Os braços batiam por coincidência documentada, não por construção.

O eixo não estava ausente do desenho. Estava *invisível*: nenhum dos três experimentos podia dizer nada sobre capacidade de modelo, porque nenhum a variou.

O piloto, e por que ele não bastava

Um piloto de US\$ 16,60 mediu 27,5 % entre Opus e Sonnet. Mas igualando por *passos*: pelos mesmos oito passos, o Opus gastou US\$ 25,18 e o Sonnet US\$ 5,53 — **4,6× mais dinheiro**. O efeito estava confundido com orçamento, exatamente como o desvio 3 do pré-registro previa antes de existir dado.

O experimento decisivo igualou por **dólar**: 36 passos de Sonnet contra 8 de Opus, os mesmos US\$ 25 por run.

15 Capacidade não se compra com orçamento

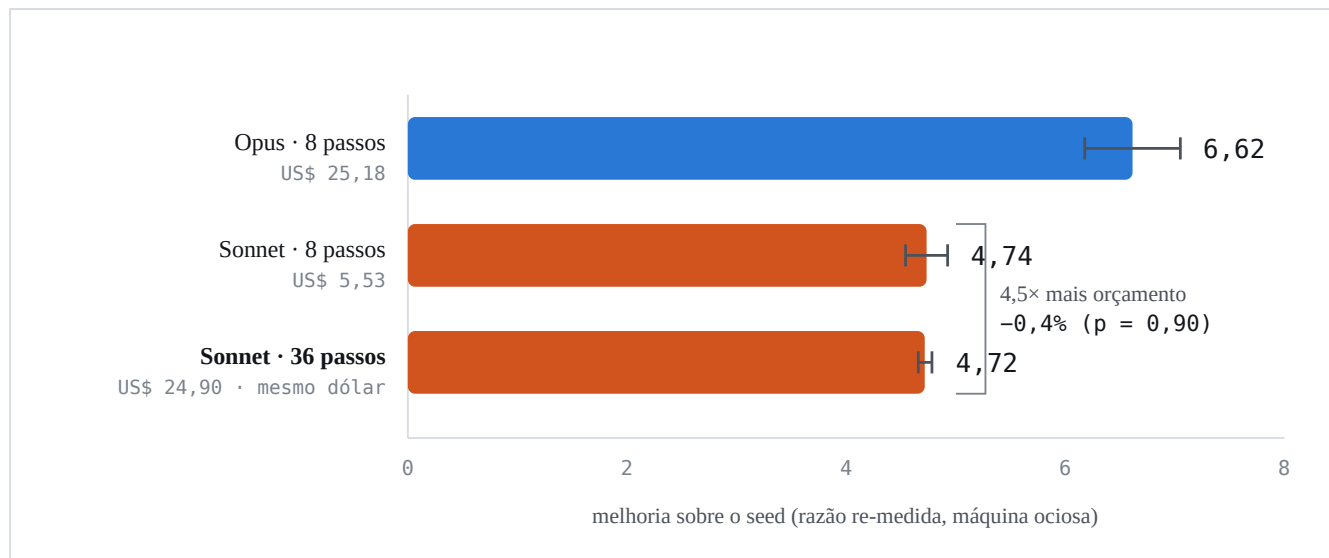


FIG. 9 Com orçamento igualado por dólar, o Sonnet fica 28,7 % abaixo do Opus. Quadruplicar seu orçamento em passos não move o resultado — 4,74 → 4,72. Barras de erro: ± 1 desvio. Valores da re-medição pareada (§ 07).

ACHADO · O ÚNICO POSITIVO

Com orçamento igualado por dólar, o Opus entrega **+28,7 %** (IC 95 % [22,9 %, 33,4 %], $p = 0,016$). Dar **4,5× mais orçamento** ao modelo menor entrega **-0,4 %** (IC 95 % [-3,6 %, +3,6 %], $p = 0,902$). Existe um teto que dinheiro não compra.

O alvo não esgotou — e a prova está no próprio experimento

O pré-registro declarou o risco: 36 passos num alvo de 8 movimentos poderiam exauri-lo, e aí o platô do Sonnet não separaria capacidade de falta de espaço. Duas evidências independentes descartam isso:

- Em duas das três sementes, o Sonnet parou de achar melhorias com **um a dois terços do orçamento por gastar** — último aceite nos passos 12, 19 e 30 de 36.
- **O Opus alcança 6,62× no mesmo alvo.** Existe headroom bem acima dos 4,72× onde o Sonnet estaciona. O platô é do modelo.

O Opus, ao contrário, ainda estava melhorando quando o orçamento acabou — último aceite no passo 7 ou 8 de 8, nas cinco sementes.

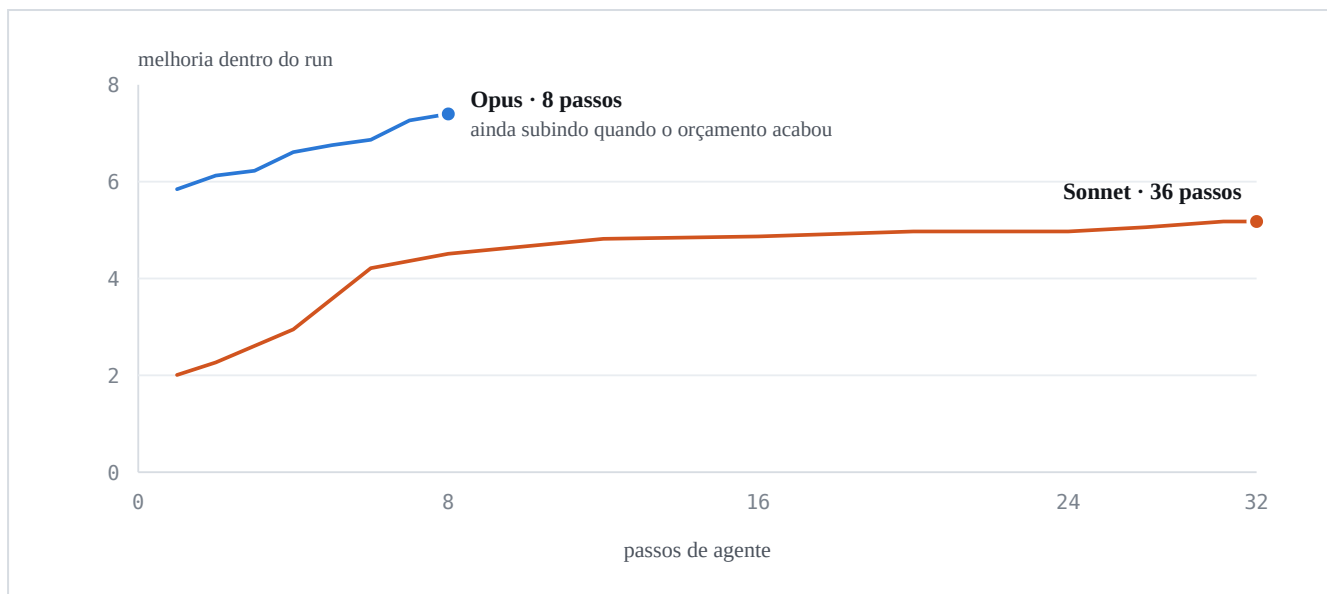


FIG. 10 A forma da busca. O Sonnet estabiliza a partir do passo ~12 e os 24 passos seguintes não movem nada; o Opus ainda subia no passo 8. **Os níveis não são comparáveis entre braços nesta figura** — a razão medida dentro do run carrega a deriva descrita em § 07. Os níveis corrigidos estão na Fig. 2; aqui o que importa é o formato.

16 Eficiência por dólar

O achado da §15 é sobre *teto*. Há uma segunda leitura, sobre *eficiência*, e ela vai na direção oposta — as duas coexistem sem contradição.

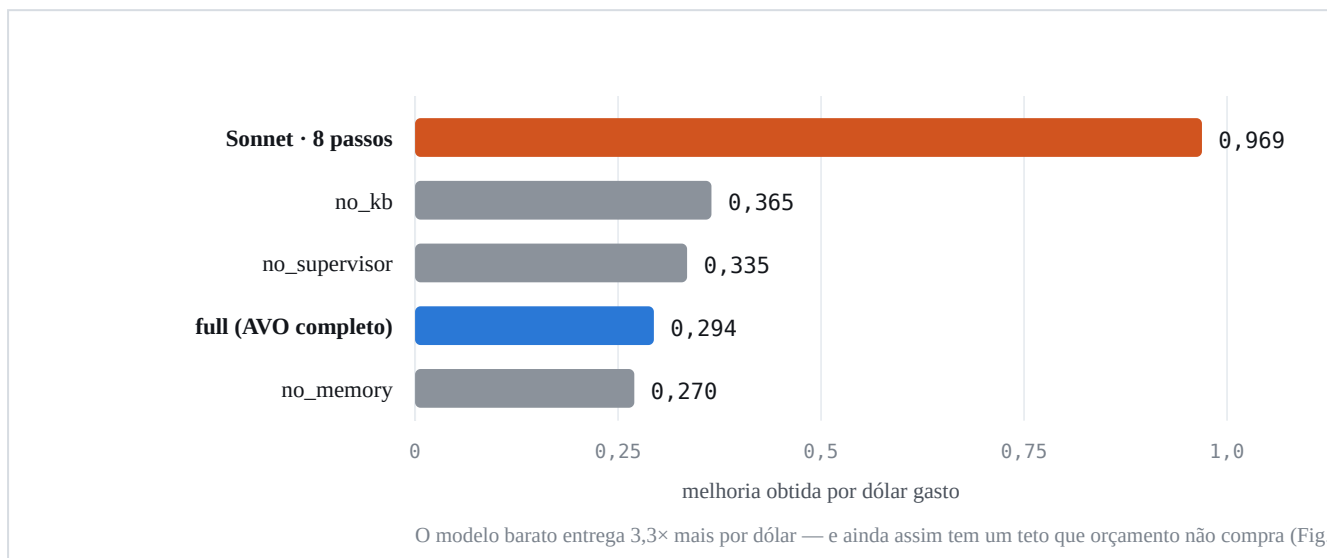


FIG. 13 Eficiência por dólar. O full não é o braço mais eficiente: no_kb e no_supervisor entregam mais por dólar porque gastam menos pelos mesmos oito passos. E o Sonnet entrega **3,3× mais que o Opus** — o que não contradiz o achado do eixo do modelo, porque eficiência e teto são coisas diferentes.

Duas observações que só aparecem quando se divide por custo:

- **O braço completo não é o mais eficiente.** no_kb e no_supervisor entregam mais por dólar, porque gastam menos pelos mesmos oito passos e chegam ao mesmo lugar. Se o componente não paga em score, ele custa em dinheiro.

- **O modelo barato entrega 3,3× mais por dólar** — e ainda assim tem um teto 28,7 % abaixo. Para uma tarefa dentro do alcance do modelo menor, ele é a escolha economicamente correta; para uma tarefa acima do teto dele, nenhum orçamento resolve.

BRAÇO	GANHO	US\$/RUN	GANHO POR US\$	ACEITOS	TURNOS/PASSO
full — AVO completo	7,396	25,18	0,294	31/40	35,6
no_kb	7,285	19,97	0,365	34/40	29,3
no_memory	7,829	29,02	0,270	27/40	39,3
no_supervisor	6,999	20,90	0,335	30/40	30,0

DESCRIPTIVO, NÃO TESTADO

O braço `no_memory` é o mais caro (US\$ 29,02) e o que menos commits aceita (27/40), com mais turnos por passo. A leitura natural é que, sem `NOTES.md`, o agente re-explora becos que já visitou — que é exatamente o que a regra de registro de becos sem saída prevê. **Mas isto é leitura de tabela descritiva com $n = 5$ e sem teste estatístico**, e as colunas secundárias não foram pré-registradas. É a hipótese mais promissora que este trabalho deixa aberta, não um resultado.

O instrumento

Um defeito de instrumento é um erro na bancada que faz o experimento medir outra coisa que não a que diz medir. Achar um vale mais que um resultado, porque o resultado passa e o defeito volta.

17 Dezoito defeitos, dois desta rodada

O repositório cataloga 18. Cinco são verificáveis linha a linha; os primeiros doze estavam descritos num relatório que **não foi commitado**, de modo que sua numeração não é auditável — e essa lacuna está registrada em vez de reconstruída de memória.

#	DEFEITO	O QUE ELE FAZIA O EXPERIMENTO MEDIR
9	Comparar um número de agora com outro de meia hora atrás	deriva de máquina, inflando tudo entre 9 % e 13 %
13	O prompt do braço barato nomeava só o <code>entrypoint</code>	viés assimétrico entre os dois braços comparados
14	Container recuperado após ~20 min ocioso	matou o piloto duas vezes; nada, mas custou relógio
15	138 diretórios temporários órfãos de processos mortos	disco a 97 %; <code>TemporaryDirectory</code> não sobrevive a <code>SIGKILL</code>
16	<code>pgrep -f "piloto.py"</code> casa com o próprio <code>grep</code>	reportar vivo um processo morto havia meia hora
17	O detector de sessão cega errava nos dois sentidos	excluiria da análise justamente quem mediu melhor
18	O denominador da métrica deriva por bloco de execução	o estado da máquina, e não o efeito do braço

Defeito 17 — o detector marcava como cega a sessão que media melhor

Uma sessão que escreve código sem nunca medir não testa o que o desenho diz testar, e sua média contamina o braço inteiro. O portão que detectava isso decidia por substring, e errava em ambas as direções. Contado sobre 300 transcrições da Ablação 2:

- **Falso positivo na tentativa.** 1.293 comandos casavam como «tentou medir», dos quais **254 eram `cat`, 227 `sed`, 126 `grep` e 70 `ls`**. Ler o fonte do avaliador contava como medir.
- **Falso negativo na medição.** As strings procuradas eram as do `stdout` *padrão*. Um agente que canaliza a saída para um parser, ou importa `medir()` para montar comparação pareada, não emite nenhuma delas: **757 comandos com saída de medição não eram reconhecidos**.

O conserto não foi alargar as strings. Foi separar duas perguntas — «o comando executa ou só lê?», resolvida por posição de comando com o corpo dos heredocs removido, e «o que voltou tem forma de medição?» — e **exigir as duas**. A forma do resultado sozinha dá **16,9 % de falso positivo**, porque `cat NOTES.md` devolve `primary = 1.16` em prosa.

CONJUNTO DE VALIDAÇÃO	ANTES	DEPOIS	ESPERADO
Ablação 2 — 160 sessões-passo	7 cegas	1 cega	6 tinham medido; a 7ª fez 0 tentativas
Piloto cego — controle negativo	6/6 cegas	6/6 cegas	tem que continuar reprovando
Fase 2A — 100 sessões	19 cegas	13 cegas	o defeito de permissão era real

O controle negativo é a parte que decide. Um detector que apenas ficasse mais permissivo aprovaria as sessões do piloto sabidamente cego — e ele continua reprovando as seis. O conserto está fixado em oito testes de regressão que cobrem as duas direções, inclusive o heredoc e a prosa com número.

Defeito 18 — o denominador deriva por bloco de execução

A métrica é `primary_final / primary_seed`, e o denominador é medido uma vez, no início do run, sobre um artefato de seed **idêntico** em todos eles.

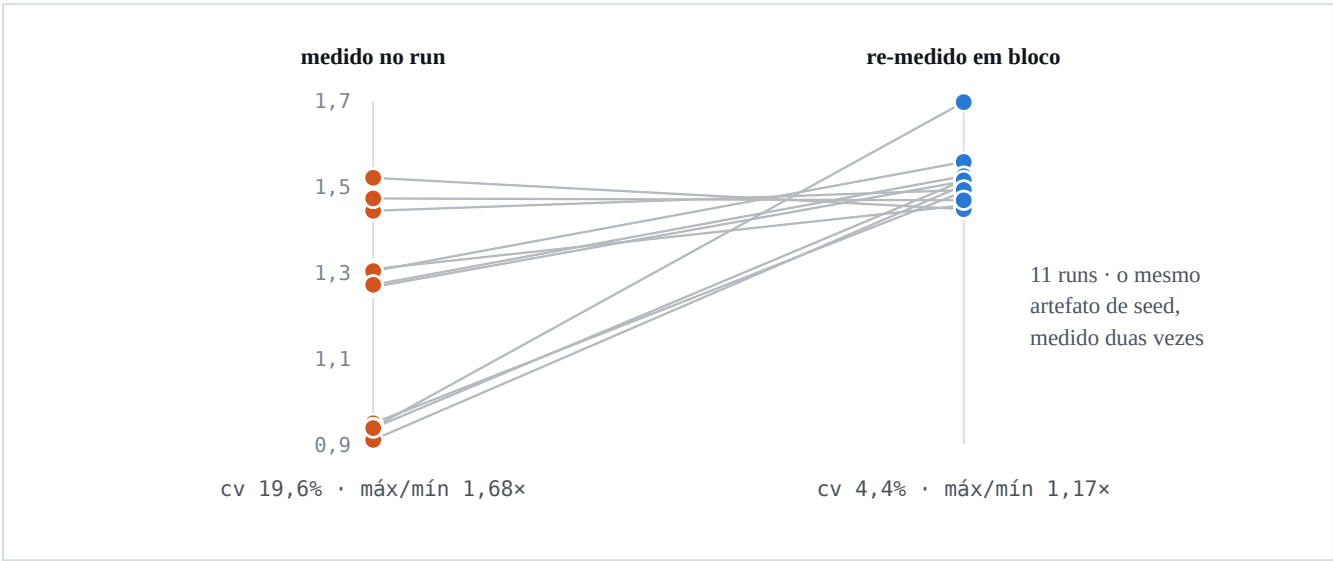


FIG. 11 O mesmo artefato de seed, medido dentro de cada run (esquerda) e re-medido em sequência imediata com a máquina ociosa (direita). A dispersão de 1,68 × colapsa para 1,17 ×. Não é ruído branco: é deriva por *bloco* de execução.

Não é ruído branco — é deriva por *bloco*: os três runs de um piloto mediram seed 0,94–0,95; os do outro braço, 0,91–1,31; os de 36 passos, 1,46–1,54. A nuance é o que importa:

- Numa ablação de braços **intercalados**, o ruído do seed é comum a todos e a razão o cancela. Testamos: fixar o denominador *piorava* o cv em três dos quatro braços. Aquela conclusão continua válida para aquele desenho.
- Entre **experimentos** de dias e durações diferentes, não há nada em comum para cancelar, e a razão passa a medir o estado da máquina.

Comparação entre experimentos exige re-medição pareada dos artefatos finais, com a máquina ociosa. Custa quatro minutos e não roda agente nenhum. Sem ela, o resultado da §15 não seria legível: os números do harness estavam inflados por fatores diferentes em cada braço.

18 A disciplina que os expôs

Os dois defeitos acima não foram achados por sorte. Foram achados porque o pré-registro obrigava a rodar o portão *antes* da análise, e porque a regra de não mexer no instrumento no meio do experimento forçou a olhar cada sessão reprovada uma a uma em vez de ajustar o detector até o número ficar bom.

Três decisões foram tomadas contra o interesse de quem as tomou, e estão registradas com data:

O adendo escrito no escuro

Escrito com 12 dos 20 runs prontos e **nenhuma estatística rodada**. Custou a exclusão de duas sementes do braço de referência.

Registrado antes de se saber que não mudaria a conclusão. Não mudou.

O instrumento deixado quebrado

A evidência do conserto do defeito 17 já estava na mão durante o experimento. Alterar um detector logo depois de ele reprovar runs do *próprio braço de referência* é o movimento que a disciplina existe para impedir, mesmo estando certo.

Consertado depois, com verificação de que a conclusão publicada não mudava.

A história bonita que não passou no teste

«O braço completo é mais consistente» — cv 4,4 % contra 6,9 / 12,0 / 15,9 %. Testada por permutação **antes de virar frase**.

p entre 0,378 e 0,603. Registrada como testada e descartada.

US\$ 74,97 dos US\$ 1.304 foram descartados por defeito de instrumento, e as pastas continuam no repositório com o nome do defeito que as invalidou — `CONTAMINADO/`, `CONFUNDIDO_EFFORT/`, `INVALIDAS/`, `CEGO/`. Um laboratório que apaga os próprios erros não pode ser auditado.

O que usar e quando

Esta parte é o produto do trabalho. As três anteriores existem para que ela possa ser escrita com números em vez de opinião.

19 A matriz de decisão

Cada linha abaixo tem um número medido atrás dela e a seção onde ele aparece. Onde não há medida, está dito que não há.

SITUAÇÃO	O QUE FAZER	EVIDÊNCIA	§
Você consegue escrever uma função objetiva verificável	Rode o loop. Incumbente + f + gate	4–7×, 51 runs	09
Você não consegue escrever essa função	Não use este método. Sem f, o laço não fecha e nada aqui se aplica	fora de escopo	21
Orçamento apertado, um único passo possível	Rode esse passo. Ele entrega a maior parte	439,6 % no passo 1	08
Decidir entre mais passos ou modelo melhor	Modelo. 4,5× mais passos no modelo menor rendeu –0,4 %	p = 0,902	15
Escolher o modelo	Teste primeiro. Foi o contraste mais barato e o único positivo	+28,7 %, US\$ 82	15
Tarefa dentro do alcance do modelo barato	Use o barato. 3,3× mais entrega por dólar	0,969 vs 0,294	16
Considerar construir knowledge base	Meça antes. Aqui não pagou, e custou US\$ 5,21/run	–1,5 %, IC cruza 0	13
Considerar memória entre passos	Talvez pague em custo, não em score. Sem ela: +15 % de custo, –13 % de aceites	descritivo, n = 5	16
Considerar supervisor de estagnação	Meça antes. Nenhum sinal em dois alvos	–5,4 % e +4,7 %	13
Construir o gate de correção	Não é opcional. Sem defesas, um memoizador marca 4.672.896	contra 4,16 do seed	05
Comparar resultados de dias diferentes	Re-meça pareado, com a máquina ociosa. 4 minutos	deriva de 1,68×	17
Planejar um experimento de ablação	Rode o piloto de potência primeiro. US\$ 35	evita US\$ 300 cegos	20

O que se paga é a **malha de feedback**: um incumbente, um número objetivo e um gate que você tentou quebrar de propósito. O que não se pagou, nesta bancada, foi o andaime cognitivo em volta dela. E o maior multiplicador disponível não estava na arquitetura — estava na escolha do modelo.

20 O preço de cada pergunta

«Não distinguível» não é «não existe». A resolução desta bancada é de ~5,5 % no melhor experimento. Andaime que valha menos que isso é invisível aqui, e nada no que medimos exclui essa possibilidade. O que se pode fazer é dizer quanto custaria enxergar.

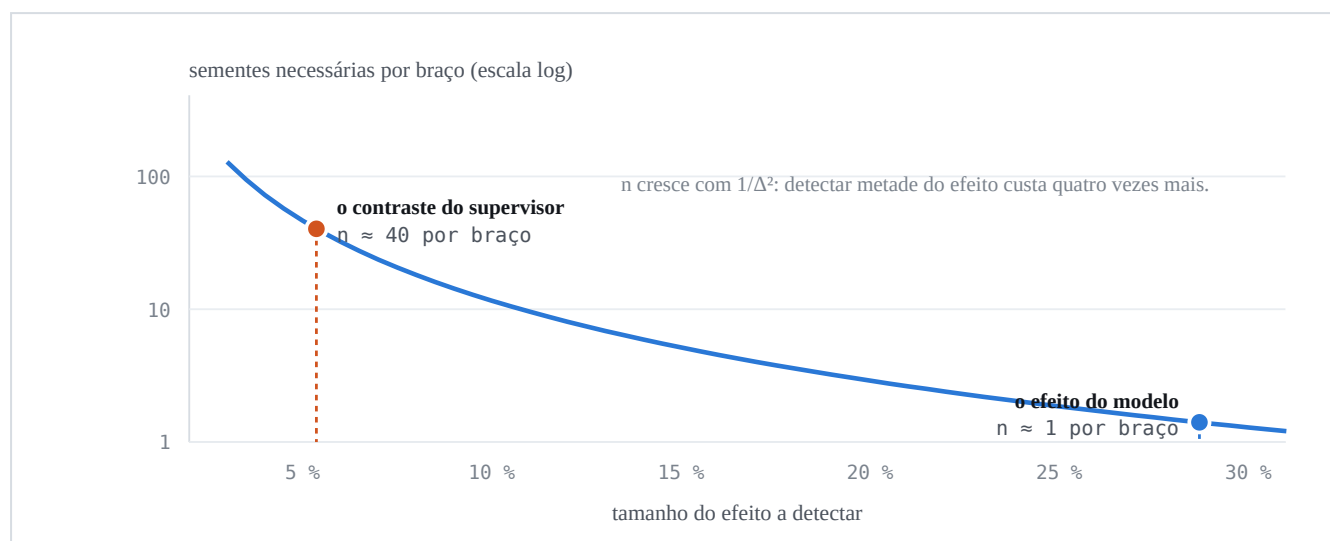


FIG. 12 O preço de cada pergunta. Como $n \approx 2 \cdot (2,8 \cdot s / \Delta)^2$, o número de sementes cresce com o inverso do quadrado do efeito. O contraste do modelo (28,7 %) sai com $n \approx 2$; o contraste do supervisor (5,4 %) custa $n \approx 41$ por braço — **US\$ 1.889 e 119 h para um único par.**

E não há poder barato. Com os dados na mão, testamos duas formas de reduzir a variância sem comprar sementes, e **as duas falharam**:

- **Parear por índice de semente.** Correlação média entre braços $r = +0,14$ — o índice não carrega sinal comum. Parear chega a *piorar* a variância da diferença em 39 % num braço.
- **Fixar o denominador da razão.** Piora em três dos quatro braços. A razão com denominador próprio já cancela a deriva dentro do run; substituí-la remove um cancelamento que estava funcionando.

A variância é estocasticidade do agente, e comprá-la é linear em n . Por isso a recomendação operacional é **rodar o piloto de potência antes de qualquer experimento**: ele custa ~US\$ 35 e diz se a pergunta é pagável, antes de US\$ 300 serem gastos numa que não é.

EXPERIMENTO	MÉDIO	US\$	RELÓGIO	RUNS	ACHOU
Ablação 1	componentes	210	15,0 h	20	nada
Fase 2A	AVO × agente simples	291	31,9 h	22	nada
Ablação 2	componentes	475	29,0 h	20	nada
Eixo do modelo	Opus × Sonnet, mesmo US\$	82	10,3 h	6	+28,7 %
Pilotos de potência	dimensionamento	67	4,3 h	13	o desenho dos acima
Descartados por defeito	—	75	—	30	os defeitos 13 e 17

21 Limitações

Esta é a seção mais importante do artigo, porque o resultado principal é negativo.

- **Potência.** $n = 3\text{--}11$ por braço; a diferença mínima detectável fica entre 5,5 % e 30 % conforme o experimento. Efeitos menores existem e seriam invisíveis aqui.
- **Orçamento igualado por passos, não por tokens**, nos três experimentos de andaime. Isso inclina o desenho *a favor* do braço completo — e mesmo assim o efeito não apareceu, o que torna o nulo mais forte.
- **Um componente não foi ablacionado** — o `no_lineage`. Declarado ausente.
- **Domínio estreito.** Otimização com `f` objetiva e verificável. Onde não se consegue escrever essa função — que é justamente onde a pergunta sobre generalidade fica interessante — nada disto foi testado.
- **Dois modelos.** A escala não está fechada; um terceiro ponto custaria ~US\$ 20.
- **O seed já funciona.** Nossos alvos partem de código correto e lento, e a §08 mostra que 90 % do ganho chega no primeiro passo. Onde o modelo sozinho vai mal — como no ARC-AGI-3, com 0,51 % de baseline — o andaime plausivelmente tem muito mais espaço para aparecer. **Isto é hipótese, não algo que medimos**, e é a explicação mais provável para a distância entre o nosso nulo e o resultado da NVIDIA.
- **As colunas secundárias não foram pré-registradas.** Custo por run, taxa de aceite e turnos por passo são descritivas; a leitura sobre a memória na §16 é a mais interessante delas e continua sendo hipótese.

22 Conclusão

O loop evolutivo com função objetiva entrega **4–7× de forma repetível** em 51 runs, em três domínios diferentes, sem que uma única trapaça tenha sido aceita. Isso é real, e é o que vale adotar: um incumbente, um número para maximizar, e um gate que você tentou quebrar de propósito.

A arquitetura em volta dele — knowledge base, memória entre passos, supervisor de estagnação, e o conjunto contra um agente sem estrutura — **não se distinguiu do ruído em três experimentos independentes, dois alvos e US\$ 976**. Não provamos que essas peças são inúteis; provamos que, ao preço que esta bancada consegue pagar, elas não aparecem. E dissemos quanto custaria enxergar: $n \approx 41$ por braço, US\$ 1.889 e 119 h para um único par.

O eixo que os três primeiros experimentos mantiveram fixo sem perceber — o modelo — foi o que moveu o resultado: **+28,7 % com o mesmo dólar**, e 4,5× mais orçamento no modelo menor não compra a diferença.

Há uma leitura mecânica que amarra as duas coisas, e ela está na §08: se 90 % do ganho chega no primeiro passo, então componentes que atuam sobre a *continuação* da busca só podem disputar os 10 % restantes — e 10 % de 7× é da mesma ordem que o ruído desta bancada. **Andaime cognitivo precisa de um problema que o modelo não resolva de primeira para ter onde aparecer.** Os nossos alvos, por serem bem-comportados, podem ser justamente os que menos favorecem a hipótese que testamos.

Para quem decide onde investir esforço de engenharia, a frase operacional é: **meça se o andaime paga antes de construí-lo, e considere que trocar o modelo pode ser o experimento mais barato que você tem.**

23 Reprodutibilidade

Todos os números deste artigo saem de `results.jsonl` e `remedido_modelo.jsonl` e reproduzem com os comandos documentados em cada relatório. O harness da busca é o upstream `gatordevin/avo`, fixado por commit em `vendor/avo.lock` — o que roda é o que foi auditado.

QUERO . . .	ARQUIVO
o resultado da ablação, com a análise pré-registrada	<code>experiments/ABLACAO_RESULTADO.md</code>
o eixo do modelo e a re-medição pareada	<code>experiments/EIXO_MODELO_RESULTADO.md</code>
os pré-registros, com os desvios declarados	<code>experiments/ABLACAO_SQL_WORKLOAD.md</code> , <code>FASE_MODELO.md</code>
os 18 defeitos de instrumento	<code>experiments/DEFEITOS.md</code>
as nove propriedades, e as falhas que as geraram	<code>docs/TARGET_DESIGN.md</code>
a bateria anti-trapaça	<code>experiments/REWARD_HACKING.md</code> , <code>tests/test_anticheat.py</code>
o que é da NVIDIA e o que é desta reprodução	<code>docs/AVO_REPLICATION_REPORT.md</code>
a escada de headroom medida à mão	<code>experiments/HEADROOM_SQL_WORKLOAD.md</code>

O relatório de replicação separa **linha a linha** o que é confirmado da NVIDIA do que é escolha desta reprodução aberta, com a fonte de cada afirmação. Nenhum design do upstream `gatordevin` é atribuído à NVIDIA, e nenhuma escolha nossa é apresentada como sendo do paper.

Sobre este documento. É a reprodução aberta de uma arquitetura publicada, feita com recursos de ordens de grandeza menores que os do trabalho original, e o resultado principal é negativo. Ele é apresentado assim de propósito: os dados brutos estão no mesmo repositório, e um artigo que afirmasse mais do que eles sustentam seria desmentido pelo primeiro leitor que abrisse um `jsonl`.

O que este trabalho contribui não é um número maior — é a comparação controlada que faltava, o preço dela calculado, e um catálogo de dezoito maneiras pelas quais uma bancada de medição pode mentir para quem a construiu.